

Comparing Deep Learning Models for Hotel Demand Forecasting and Dynamic Room Pricing

Jack DeMarinis
CSC 561 Final Project
United States

Abstract—This project develops an end-to-end hotel demand forecasting and pricing-support pipeline using real operational hotel data. Raw reservation, room-sales, and forecast reports are cleaned, deduplicated, audited, and converted into a daily modeling table covering 2023–2025. The main forecasting target is daily rooms sold. Classical baselines, tree-based tabular models, and neural sequence models are evaluated using chronological train, validation, and test splits. The primary reported model convention excludes operational rows flagged as known anomalies. XGBoost is the best model by test mean absolute error (MAE), achieving MAE 7.443, RMSE 9.927, and R^2 of 0.585 on the unflagged 2025 test period. Deep learning models are competitive but do not surpass the best tree models: the best GRU reaches MAE 7.813, and the best Transformer reaches MAE 7.835. A bounded pricing layer converts the selected XGBoost demand forecast into aggressive and conservative ADR recommendations. Under a static counterfactual that holds realized rooms sold fixed, the aggressive scenario increases simulated unflagged 2025 room revenue by \$88,101.38 and RevPAR by \$4.15, while the conservative scenario increases simulated revenue by \$44,092.35 and RevPAR by \$2.08. The results support a practical MVP conclusion: tree-based tabular forecasting remains the strongest approach for this small daily dataset, while GRU and Transformer models provide useful deep learning comparisons.

Index Terms—hotel demand forecasting, revenue management, XGBoost, random forest, GRU, LSTM, Transformer, dynamic pricing

I. Introduction

Hotels make pricing decisions under uncertainty: demand varies by day of week, season, events, booking behavior, room availability, and prior occupancy. Many small or independent hotel operations still rely on spreadsheets, manual review, or fixed pricing rules, even though commercial revenue management systems use data-driven demand signals to update prices more systematically. The goal of this project is to build an academic minimum viable product (MVP) that forecasts daily hotel demand and converts that forecast into bounded pricing recommendations.

The project is motivated by a real business setting using historical operational hotel reports. From a machine learning perspective, the task is a structured time-series regression problem. From a business perspective, it is a pricing-support problem where prediction errors and recommendation constraints both matter. The project therefore evaluates predictive performance using MAE, RMSE, MAPE, and R^2 , and evaluates pricing support

using static revenue, ADR, occupancy, and RevPAR metrics.

The proposal emphasized comparison between classical machine learning and deep learning architectures. This report implements that comparison using simple time-series baselines, Ridge regression, Random Forests [1], XGBoost [2], MLP models, LSTM models [3], GRU models [4], and small Transformer encoders [5]. The final system uses a two-stage pipeline: first forecast demand, then apply bounded pricing rules.

II. Data Pipeline

The raw data consist of operational workbooks, including reservation-style guest reports, Block RoomSales reports, and ReservationForecast reports. The final cleaned datasets are stored under `cleaned_data/`, while derived modeling outputs are stored under `processed_data/` and `reports/`.

The final modeling table is `processed_data/modeling_daily.csv`. It contains 1,096 daily rows and 209 columns covering January 1, 2023 through December 31, 2025. The table is date-grain: each row represents one business date. It joins cleaned room-sales history, reservation forecast fields, and guest-arrival enrichment. The primary target is `target_rooms_sold`.

Cleaning included parsing multi-line guest report blocks, standardizing dates and numeric fields, normalizing key text values, preserving source lineage, and removing duplicates with an auditable deduplication process. The data quality audit flagged two over-capacity rows and twelve zero-room-sold rows. These rows are retained for traceability, but the primary reported model convention excludes all flagged dates because they are known operational anomalies.

Fig. 1 summarizes the final pipeline.

III. Feature Engineering

Features were designed to be forecast-oriented and leakage-safe. Same-day operational fields, same-day guest enrichment fields, and raw future ReservationForecast fields were excluded from learned model inputs. The model inputs combine:

- calendar features such as day of week, month, quarter, weekend flags, and cyclical encodings;

Hotel Demand Forecasting and Pricing MVP Pipeline

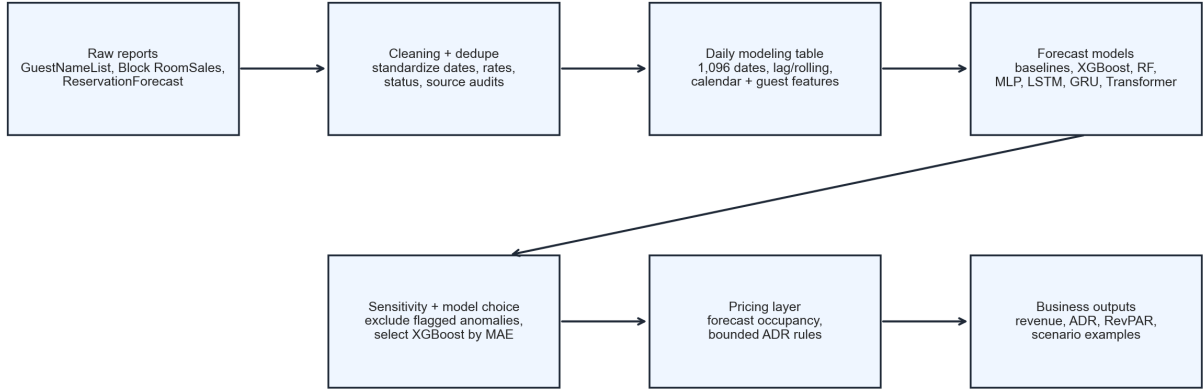


Fig. 1. End-to-end hotel demand forecasting and pricing MVP pipeline.

- lagged historical values for rooms sold, ADR, revenue, and occupancy;
- rolling means, medians, and standard deviations over 7, 14, 28, and 56 day windows;
- derived comparisons such as week-over-week changes.

All rolling features are shifted so that the current day's target is not included in the feature values. This matters because the intended use case is forecasting before final realized demand is known.

IV. Models and Evaluation

A. Splitting Strategy

The dataset is split chronologically. Training uses January 1, 2023 through June 30, 2024; validation uses July 1, 2024 through December 31, 2024; and testing uses calendar year 2025. This matches the operational goal of generalizing to a later time period.

Three data-quality conventions are reported. The full-data reference uses all rows. The over-capacity sensitivity removes only rows where rooms sold exceeded inferred capacity. The primary convention, `exclude_all_flagged`, excludes all current operational anomaly flags.

B. Classical and Tree-Based Models

Baselines include previous-day demand, previous-week demand, rolling averages, and Ridge regression. Tree-based tabular models include Random Forest and XGBoost. The implementation uses scikit-learn [6] and XGBoost.

Table I shows the full-data 2025 test results. XGBoost is best by MAE, while Random Forest has slightly stronger RMSE and R^2 .

TABLE I
Full-Data 2025 Test Metrics

Model	Rows	MAE	RMSE	R^2
XGBoost	365	8.039	11.210	0.539
Random Forest	365	8.062	10.870	0.567
Previous Week	365	9.551	13.438	0.338
Previous Day	365	11.307	15.987	0.063
Rolling 7d	365	12.528	15.107	0.164
Ridge	365	12.744	14.938	0.182
Rolling 28d	365	13.511	16.073	0.053

TABLE II
Best Learned Model by Data-Quality Convention

Scenario	Model	Rows	MAE	RMSE	R^2
All flags excl.	XGBoost	354	7.443	9.927	0.585
Over-cap excl.	RF	365	8.019	10.757	0.576
Full data	XGBoost	365	8.039	11.210	0.539

Table II shows the sensitivity analysis. The primary report result is XGBoost under `exclude_all_flagged`: MAE 7.443, RMSE 9.927, and R^2 0.585.

C. Deep Learning Models

The deep learning comparison includes tabular MLP models and sequence models. LSTM, GRU, and Transformer candidates use sequential windows ending on each prediction date. The sweep includes lookback windows of 14, 28, 42, 56, and 84 days. PyTorch [7] is used for LSTM, GRU, and Transformer training, with AdamW optimization [8], Smooth L1 loss, gradient clipping, and early stopping on validation loss.

TABLE III
Tree Models vs. Deep Learning on Unflagged 2025 Test Rows

Model	Rows	MAE	RMSE	R^2
XGBoost	354	7.443	9.927	0.585
Random Forest	354	7.466	9.834	0.593
GRU 56d wide	354	7.813	10.588	0.528
GRU 56d regularized	354	7.822	10.616	0.525
Transformer 28d	354	7.835	10.305	0.553
GRU 42d	354	7.937	10.587	0.528
Transformer 56d	354	7.945	10.561	0.530

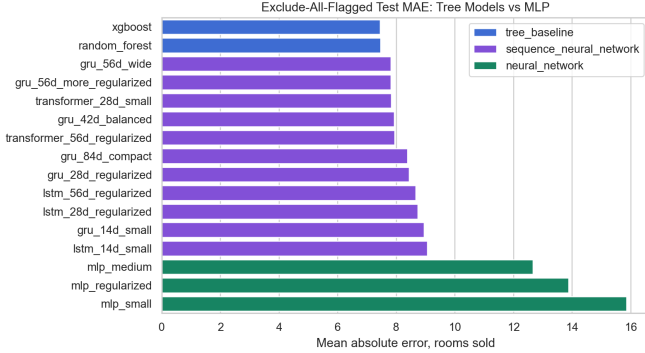


Fig. 2. Exclude-all-flagged test MAE comparison across tree and neural models.

Table III compares the strongest models under the primary exclude_all_flagged test convention. The best GRU reaches MAE 7.813, and the best Transformer reaches MAE 7.835. These sequence models get close to the tree models, but XGBoost remains best by MAE and Random Forest remains best by RMSE and R^2 .

Fig. 2 visualizes the model comparison. The main interpretation is that sequence neural networks are viable for this task, but the dataset is small enough that tree-based tabular models still generalize slightly better.

V. Feature Importance

The XGBoost feature-importance analysis shows that recent weekly history is the strongest signal. The top features are target_rooms_sold_lag_7d, target_rooms_sold_lag_14d, block_room_revenue_net_lag_7d, day-of-week encoding, and one-day lagged room revenue. This is consistent with the domain expectation that hotel demand has strong weekly seasonality and short-term persistence.

VI. Pricing Recommendation Layer

The pricing layer uses the selected XGBoost forecast. Forecast occupancy is computed as forecast rooms sold divided by total property rooms. A reference ADR is chosen from historical same-day ADR with rolling and lagged ADR fallbacks. The pricing rule then applies bounded demand-based adjustments.

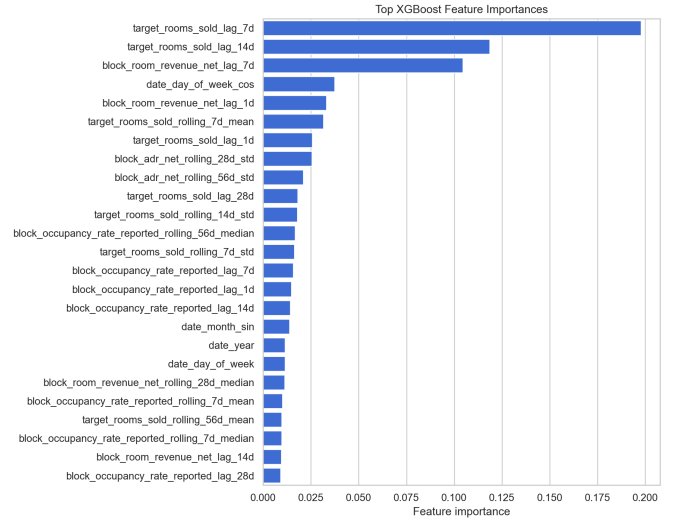


Fig. 3. Top XGBoost feature importances under the primary training setup.

TABLE IV
Static Pricing Scenario Results on Unflagged 2025 Test Days

Scenario	Hist. Rev.	Sim. Rev.	Delta	Delta %
Aggressive	\$2.018M	\$2.106M	\$88.1K	4.37%
Conservative	\$2.018M	\$2.062M	\$44.1K	2.18%

Two scenarios are evaluated. The aggressive scenario uses 4%, 8%, and 12% adjustment bands. The conservative scenario uses 2%, 4%, and 6% bands. ADR is also bounded by minimum and maximum ADR constraints and by a maximum daily change from the reference ADR.

The pricing simulation is intentionally static. It applies recommended ADR to actual realized rooms sold and does not estimate price elasticity, competitor reaction, or occupancy changes caused by price changes. This makes the result a useful academic counterfactual, not proof of actual achievable revenue.

Table IV shows static unflagged 2025 pricing results. The aggressive scenario produces a simulated revenue lift of \$88,101.38, while the conservative scenario produces \$44,092.35.

Table V adds occupancy and RevPAR context. Static occupancy delta is zero by design because rooms sold are held fixed. RevPAR improves by \$4.15 in the aggressive scenario and \$2.08 in the conservative scenario.

Fig. 4 compares historical and recommended ADR over time. Fig. 5 compares the aggressive and conservative revenue deltas.

VII. High-Demand Scenario Examples

To make the pricing recommendations concrete, Table VI shows high-demand weekend dates from the unflagged 2025 test period. These dates have forecast occupancy near sellout levels and therefore receive upward

TABLE V
Business Metrics on Unflagged 2025 Test Days

Scenario	Act. Occ.	Fcst. Occ.	Hist. RevPAR	Sim. RevPAR
Aggressive	63.63%	66.13%	\$95.01	\$99.16
Conservative	63.63%	66.13%	\$95.01	\$97.09

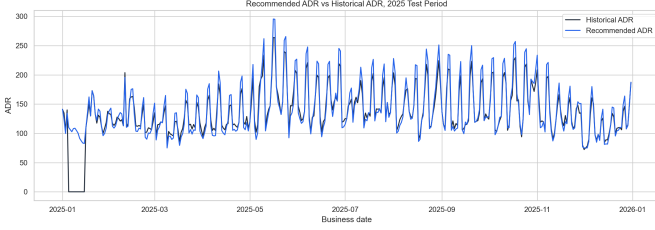


Fig. 4. Recommended ADR versus historical ADR during the 2025 test period.

ADR recommendations. For example, October 25, 2025 had 59 actual rooms sold and a forecast of 60.0 rooms; the aggressive rule recommends increasing ADR from \$218.36 to \$244.56, which yields a static simulated revenue delta of \$1,545.61.

VIII. Limitations

There are several limitations. First, the pricing simulation does not model demand response to price changes. A real revenue management system would need elasticity estimates, competitor pricing, channel mix, and customer response modeling. Second, some proposal features such as competitor prices and event indicators were only included if available; the final dataset did not contain a reliable external competitor or event feed. Third, January 2025 zero-room-sold dates were flagged as likely operational anomalies and should not be used as primary business evidence without verification. Fourth, the dataset is small for deep learning. The best GRU and Transformer models get close, but the tabular tree models remain better suited to the available sample size.

IX. Conclusion

This project implements the proposed hotel demand forecasting and pricing-support MVP end to end. The final system cleans operational reports, builds a daily modeling table, audits data quality, compares classical baselines against deep learning models, selects a primary demand model, and converts forecasts into bounded pricing recommendations.

The strongest forecasting result is XGBoost under the `exclude_all_flagged` convention, with test MAE 7.443, RMSE 9.927, and R^2 0.585. Deep learning models provide a meaningful comparison: the best GRU reaches MAE 7.813 and the best Transformer reaches MAE 7.835, close to but not better than XGBoost. The pricing layer shows practical business potential under static assumptions, with simulated unflagged 2025 revenue lifts of 4.37% for

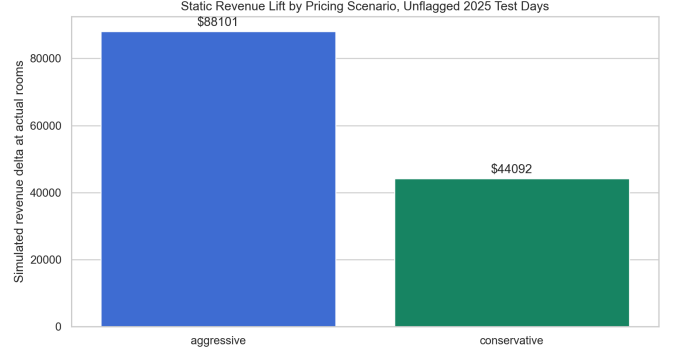


Fig. 5. Static simulated revenue lift by pricing scenario.

the aggressive scenario and 2.18% for the conservative scenario.

Overall, the final recommendation is to use XGBoost as the MVP forecasting model, present GRU and Transformer results as strong deep learning comparisons, and treat the pricing layer as a bounded decision-support tool rather than a complete commercial revenue management system.

Appendix A

Repository Reproducibility Notes

This appendix is written as an audit trail for the code and artifacts in the project repository rather than as an additional results section. The main paper reports the most important model and pricing outcomes. The appendix records how those outcomes are produced, what assumptions were made during cleaning, and which files in the repository correspond to each stage of the pipeline. This follows the purpose of appendices in many machine-learning papers: details that are too operational for the main narrative are preserved so that an evaluator can reproduce and critique the experiment.

A. Repository Organization

The repository is organized around a simple experimental workflow. Raw operational workbooks are stored in `Data/`. The original project proposal and assignment files are stored in `Documents/`. Cleaned outputs are written to `cleaned_data/`; derived modeling features are written to `processed_data/`; final metrics, figures, summaries, and the IEEE report are written under `reports/`. The source code is split by responsibility under `src/`: feature construction, model training, evaluation, and pricing. This organization matters because the report is not a hand-written result summary disconnected from the code; the tables and figures in the paper are generated from the same CSV and JSON artifacts used by the scripts.

The main script entry points are as follows. `scripts/clean_new_operational_reports.py` parses the raw hotel report exports and creates the cleaned operational datasets.

TABLE VI
High-Demand Weekend Pricing Examples

Date	Day	Actual	Forecast	Hist. ADR	Agg. ADR	Cons. ADR	Agg. Delta
2025-10-25	Saturday	59	60.0	\$218.36	\$244.56	\$231.46	\$1,545.61
2025-08-09	Saturday	57	60.0	\$173.94	\$194.81	\$184.38	\$1,189.38
2025-11-15	Saturday	59	60.0	\$166.05	\$185.98	\$176.01	\$1,176.15
2025-12-06	Saturday	60	60.0	\$160.63	\$179.91	\$170.27	\$1,156.63
2025-11-22	Saturday	58	60.0	\$161.24	\$180.59	\$170.91	\$1,122.11
2025-05-24	Saturday	57	59.9	\$237.02	\$265.46	\$251.24	\$1,621.31

src/features/build_guest_daily_enrichment.py converts cleaned reservation-style guest records into daily arrival, room-type, market-segment, and stay-proxy features. src/features/build_daily_modeling_table.py joins the cleaned daily Block RoomSales data, ReservationForecast data, and guest enrichment into processed_data/modeling_daily.csv. src/evaluation/audit_modeling_data_quality.py creates anomaly-review records. src/models/train_baselines.py, src/evaluation/run_baseline_sensitivity.py, and src/models/train_deep_learning.py generate the forecasting comparisons. src/pricing/build_pricing_recommendations.py builds ADR recommendations and static revenue simulations. Finally, src/evaluation/build_final_report_artifacts.py and src/evaluation/build_report_summary.py generate the final report support files.

The generated report folder is deliberately separated into subfolders. reports/audit/ contains data-quality flags; reports/baselines/ contains baseline and sensitivity predictions and metrics; reports/deep_learning/ contains neural-model predictions and comparison files; reports/pricing/ contains the ADR recommendation outputs and business metrics; reports/figures/ contains PNG figures; and reports/final/ contains the IEEE L^AT_EX source, bibliography, PDF, and generated project summary. The organization is intended to make it possible to trace every claim in the paper back to a concrete artifact without scanning a flat report folder.

B. Reproduction Order

The expected reproduction order is pipeline-first, then model-first, then report-first. The cleaned data should be regenerated before modeling if any raw report file changes. After cleaning, the daily enrichment and modeling table should be rebuilt before any metrics are trusted. The report should be rebuilt only after the modeling and pricing artifacts have been refreshed.

The practical order is:

- clean the operational exports into cleaned_data/;
- build guest daily enrichment into processed_data/guest_daily_enrichment.csv;
- build the final date-grain modeling table into processed_data/modeling_daily.csv;

- audit modeling rows and write reports/audit/data_quality_flags.csv;
- train baseline and learned tabular models;
- run sensitivity analysis under the three data-quality conventions;
- train the neural comparison models;
- build pricing recommendations and report figures;
- compile reports/final/final_report_ieee.tex.

The final modeling table is the central contract between the data work and the model work. It has one row per business date, 1,096 rows, and 209 columns covering January 1, 2023 through December 31, 2025. It has no missing business dates and no duplicate business dates. The forecasting target is target_rooms_sold. The model split is chronological: training through June 30, 2024; validation from July 1, 2024 through December 31, 2024; and testing on calendar year 2025. The main paper uses 2025 as the held-out evaluation period because it is the only full future year after the 2023–2024 training and validation history.

Appendix B Data Processing Details

A. Block RoomSales Processing

The Block RoomSales exports are the most important operational source because they provide continuous daily property-level rooms sold, revenue, ADR, RevPAR, inventory, out-of-order rooms, transient rooms, and group rooms from 2023 through 2025. The cleaned Block family contains 1,096 daily property-total rows, 2,192 room-class rows, 10,960 room-type rows, and 10 special adjustment rows. The inferred hotel inventory is consistently 60 rooms across all Block files.

The raw Block workbooks are not simple flat files. The cleaning code treats date rows as property-level totals, Standard and Suite rows as room-class rollups, and ten room-type codes as leaf room types. Rows labeled *** are preserved separately as special adjustment rows rather than mixed into room type rows. This distinction prevents double counting: room-class rows summarize room types, while adjustment rows can carry revenue-only corrections that should not be interpreted as normal room inventory. Summary sections and report footnotes are excluded from the cleaned analytical tables.

The Block processing includes reconciliation checks. Room-type rows plus deduplicated adjustment rows reconcile to property daily totals for rooms sold, transient rooms, group rooms, day-use rooms, comp rooms, out-of-order rooms, on-market rooms, gross revenue, and net revenue. The cleaning summary also confirms that each business date has exactly ten leaf room-type rows. Those checks are important because the final forecast target comes from this source family.

B. ReservationForecast Processing

The ReservationForecast files cover 2023 and 2024 with 731 daily observations and no missing dates. They provide arrivals, departures, stayovers, rooms sold, revenue, ADR, RevPAR, transient rooms, and group-specific fields. These reports are useful for validation and enrichment because they overlap fully with the 2023–2024 Block RoomSales history. On overlapping dates, rooms sold, out-of-order rooms, and net room revenue match exactly against the Block family.

One important caveat is the occupancy denominator. The cleaned summaries show that ReservationForecast occupancy is much closer to rooms sold divided by on-market rooms, while Block RoomSales occupancy is based on total property inventory. The final modeling table therefore keeps occupancy-definition columns that make the difference explicit, rather than forcing the two reports into a single ambiguous occupancy field. This matters for interpretation because a day with out-of-order rooms can have different reported occupancy depending on whether the denominator is total rooms or sellable rooms.

C. Guest-List Processing and Privacy

The GuestNameList exports are reservation-style reports for 2025. They are not one-row-per-stay tables. Each logical reservation appears as a multi-line block with report-specific row structure. The parser converts these blocks into 18,365 provisional records and then deduplicates them into 10,579 cleaned stay-like records. The duplicate process removes 7,786 duplicate rows across 5,348 duplicate groups, keeping the most complete copy and preserving source audit fields.

The guest-list data contain personally identifiable information, including guest names, emails, comments, confirmation numbers, folio numbers, and room numbers. These fields are useful for cleaning and deduplication, but they are not used as forecasting features. The daily enrichment output explicitly excludes those PII columns. Instead, it aggregates into daily counts and rates such as arrivals by status, market-segment counts and shares, room-type arrival shares, average booked rate, average nights, and stay room-night proxies. This design keeps the model focused on operational signals while avoiding direct use of guest identity information.

The guest-derived features are also treated cautiously because the guest-list source mainly covers 2025. They are

useful for context and final test-period diagnostics, but they are not treated as a complete multi-year historical signal for the baseline forecasting task. This is one reason the learned model feature selector excludes same-day guest-enrichment fields from forecasting inputs.

Appendix C

Modeling, Leakage Control, and Sensitivity

A. Feature Policy

The forecasting task is set up as a daily time-series regression problem. The model may use calendar features and historical shifted signals, but not same-day operational outcomes that would be unavailable when forecasting future demand. The feature groups include date fields, cyclical encodings for day of week, month, and day of year, lag features at 1, 7, 14, 28, and 365 days, and rolling windows over 7, 14, 28, and 56 days. Historical base columns include rooms sold, net ADR, net room revenue, and reported occupancy.

This policy is implemented in `src/models/train_baselines.py`. Direct baselines use shifted target history only. Learned tabular models use selected date and historical lag/rolling fields. Same-day Block fields, ReservationForecast fields, source lineage columns, and audit fields are blocked unless they appear only as lagged or rolling historical versions. This is the most important experimental safeguard in the repository because leakage would otherwise make the hotel forecast look artificially strong.

B. Model Families

The experiment compares simple historical baselines, Ridge regression, Random Forest, XGBoost, MLP variants, and sequence neural networks. The direct baselines represent what a hotel analyst could do without machine learning: yesterday’s demand, the same weekday last week, and short rolling averages. Ridge regression provides a linear learned reference. Random Forest and XGBoost are the strongest tabular approaches for the available sample size. The MLP, LSTM, GRU, and Transformer variants test the project proposal’s deep learning hypothesis.

The neural models are intentionally modest. The sequence models use lookback windows from 14 to 84 days, Smooth L1 loss, AdamW optimization, gradient clipping, and validation-loss early stopping. This avoids presenting an oversized neural architecture for a dataset with only three years of daily records. The final result is therefore not that deep learning is impossible for hotel forecasting. The narrower conclusion is that, for this repository’s current data volume and feature set, tree-based tabular models generalize better than the tested neural sequence models.

C. Data-Quality Sensitivity

The anomaly audit writes review records rather than modifying the source data. Fourteen rows are flagged in the full modeling table: twelve zero-room-sold rows and

two rows where rooms sold exceed the 60-room property inventory. The primary convention excludes all flagged rows, which leaves 354 test rows in 2025. The sensitivity convention that excludes only over-capacity rows leaves 365 test rows because the zero-room-sold dates remain in the test period. The full-data convention also uses 365 test rows.

The decision-level sensitivity result is stable. Under the primary exclude-all-flagged convention, XGBoost is the best learned model by MAE with 7.443. Under the over-capacity-only convention, Random Forest is best by MAE with 8.019. Under the full-data reference convention, XGBoost is again best by MAE with 8.039. These differences justify two choices in the main report. First, the primary result should use the exclude-all-flagged convention because it avoids allowing known operational anomalies to dominate the business interpretation. Second, Random Forest should remain visible in the comparison because it is competitive and sometimes stronger by RMSE or R^2 .

The deep learning comparison is also stable in interpretation. The best neural model is `gru_56d_wide`, with MAE 7.813 on the unflagged 2025 test period. The best Transformer reaches MAE 7.835. These results are close enough to show that sequence models are meaningful, but not close enough to replace XGBoost as the recommended MVP model. The weaker MLP results also suggest that simply adding neural-network capacity to tabular features is not sufficient for this small dataset.

Appendix D

Pricing Appendix and Deployment Notes

A. Pricing Rule Rationale

The pricing layer is a bounded decision-support rule, not a complete revenue management optimizer. It starts with the selected XGBoost forecast, converts forecast rooms sold into forecast occupancy using the 60-room inventory, assigns a demand band, and adjusts a reference ADR. The reference ADR is chosen from historical same-day net ADR when available, then rolling or lagged ADR fallbacks, and finally a median nonzero ADR fallback. The recommended ADR is bounded by a minimum ADR, maximum ADR, and maximum 20% daily movement from the reference ADR.

Two scenarios are reported because a single pricing curve would imply false precision. The aggressive scenario uses larger upward and downward movements, while the conservative scenario uses smaller movements. The aggressive rule produces a 4.37% static revenue lift on unflagged 2025 test days, and the conservative rule produces a 2.18% lift. These are static counterfactuals: they apply recommended ADR to actual realized rooms sold. They do not estimate whether customers would still book at the higher price, whether competitors would react, or whether channel mix would change.

B. Recommended Production Extensions

The current system is suitable as an academic MVP and as a prototype for hotel pricing support, but not as an automated pricing engine. A production version should add at least four extensions. First, it should estimate price elasticity or use controlled experiments so that ADR changes affect expected rooms sold instead of being applied to fixed historical occupancy. Second, it should add competitor-rate, local-event, and holiday-demand feeds. Third, it should monitor forecast errors by day of week, occupancy band, month, and high-demand periods so the model can be recalibrated before systematic misses affect pricing. Fourth, it should require human approval for flagged dates, unusually high ADR changes, and dates near sellout.

The current recommendation should therefore be interpreted conservatively. The model can identify days when historical demand signals suggest higher or lower occupancy pressure. It can translate that signal into bounded ADR advice. It cannot yet answer whether a specific ADR change would maximize profit after accounting for customer response, cancellation behavior, competitor movement, or distribution costs.

Appendix E

Artifact Traceability

The major report claims map to files in the repository. The final modeling table is `processed_data/modeling_daily.csv`. Cleaning summaries are stored as JSON files in `cleaned_data/` and `processed_data/`. Baseline and sensitivity metrics are stored in `reports/baselines/`. Deep learning metrics are stored in `reports/deep_learning/`. Pricing recommendations, business metrics, and high-demand examples are stored in `reports/pricing/`. Figures are stored in `reports/figures/`. The final $\text{\LaTeX}$ source and PDF are stored in `reports/final/`.

The most important audit artifacts are the data-quality flags, baseline sensitivity metrics, deep-learning comparison metrics, pricing scenario comparison, and pricing business-metric comparison. These live in the audit, baselines, deep_learning, and pricing report subfolders. Those files are the quickest way to check whether the report has drifted from the current code output. If the report is regenerated after changing the pipeline, these artifacts should be inspected before the PDF is treated as final.

The appendix also records two deliberate exclusions. First, raw PII is not included in modeling artifacts. Guest identity fields remain useful only for cleaning and traceability. Second, the raw STAR monthly benchmarking files are not yet integrated into the forecasting model. They are potentially useful future external features, but the current MVP relies on operational property data rather than competitor benchmarking feeds. These exclusions reduce the feature set, but they make the final experiment

more defensible because every included signal has clear availability and source lineage.

References

- [1] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [2] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [3] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [4] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, “Learning phrase representations using RNN encoder–decoder for statistical machine translation,” in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, 2014, pp. 1724–1734.
- [5] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [6] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, “Scikit-learn: Machine learning in python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [7] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimeshine, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems*, vol. 32, 2019.
- [8] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *International Conference on Learning Representations*, 2019.